

A grandes rasgos, el calendario oficial del proyecto está estructurado en **11 sprints semanales (del Sprint 0 al Sprint 10)**. Sigue una metodología ágil e incremental, donde ninguna tecnología se aprende de forma aislada. Cada semana los alumnos conectan el almacenamiento de datos (\$PostgreSQL\$), la lógica de negocio (\$Node.js/Express\$) y la interfaz de usuario (\$ReactJS\$) para dar vida a un Mínimo Producto Viable (MVP) real y funcional.

La evolución del proyecto se divide en **cuatro grandes fases operativas**:

Fase 1: Diseño, Modelado y Setup (Semana 21)

- **Sprint 0 (Arquitectura y Modelado):** Es la fase de diseño conceptual. Los equipos definen el alcance de su MVP, diseñan el Diagrama Entidad-Relación (DER), valoran las restricciones de los atributos en el [README.md](#) y preparan su entorno local aislado utilizando contenedores de Docker con PostgreSQL y pgAdmin.

Fase 2: Desarrollo del Backend y Base de Datos (Semanas 22 a 25)

- **Sprint 1 (Script de Datos e Inicio del API):** Se ejecuta la creación física de las tablas en el contenedor mediante archivos [.sql](#). Se introducen los *seeders* con datos falsos iniciales y se monta la estructura limpia de carpetas en Node.js siguiendo el patrón MVC.
- **Sprint 2 (Rutas y Conexión SQL - CRUD):** El backend interactúa de forma real con la base de datos a través de un pool de conexiones. Se desarrollan y validan con Postman o Thunder Client los endpoints que permiten crear, leer, actualizar y borrar (CRUD) las entidades base del negocio.
- **Sprint 3 (Autenticación, Seguridad y Lógica Avanzada):** Se implementa el registro y login seguro con contraseñas encriptadas con [bcrypt](#). Se protegen los recursos privados mediante JSON Web Tokens (JWT) y se aplican Transacciones SQL avanzadas en los procesos críticos que comprometan la integridad de los datos.
- **Sprint 4 (Casos de Uso Especiales del Backend):** Se finaliza la lógica del servidor resolviendo las necesidades complejas y específicas de cada aplicación (ej. ingesta de APIs externas en formato [JSONB](#), subida de archivos multimedia con [multer](#) o tareas automatizadas con [node-cron](#)).

Fase 3: Desarrollo del Frontend e Integración (Semanas 26 a 28)

- **Sprint 5 (Despegue del Frontend - React Base):** Inicialización de la interfaz de usuario en React con Vite. Se configura el enrutamiento y las vistas protegidas mediante [react-router-dom](#) y se realiza la maquetación modular con componentes reutilizables utilizando datos estáticos.
- **Sprint 6 (Conexión Fullstack - Consumo del API):** Se une el frontend con el backend mediante llamadas asíncronas ([fetch](#) o [axios](#)). Se gestionan los formularios enviando datos reales a la base de datos y se persiste de forma segura el token JWT en el cliente.

- **Sprint 7 (Tiempo Real y Librerías Especializadas):** Se añade la interactividad avanzada y la propuesta de valor de cada grupo , implementando herramientas como WebSockets ([Socket.io](#)) para sincronización síncrona, mapas de calor con librerías de gráficos o mapas geolocalizados interactivos.

Fase 4: Despliegue, Calidad y Defensa (Semanas 29 a 31)

- **Sprint 8 (Despliegue en Producción):** Filosofía Cloud-First. La aplicación sale del entorno local migrando la base de datos a la nube (Supabase/ElephantSQL) , desplegando el backend (Render/Railway) y el frontend (Vercel/Netlify) bajo una gestión estricta de variables de entorno [.env](#).
- **Sprint 9 (Testing, Refactor y QA):** Fase de aseguramiento de calidad orientada a "romper" la aplicación en producción para estabilizarla. Se corrigen bugs, se pulen las interfaces con estados de carga/error claros y se realiza una auditoría de código contra vulnerabilidades básicas como SQL Injection.
- **Sprint 10 (Ensayos de Defensa y Presentación):** Preparación del guion técnico enfocado en las competencias profesionales del módulo y ensayos de la demo técnica en vivo con pantallas reales. Todo culmina el **29/07** con la defensa oficial en directo de la aplicación desplegada y funcionando en producción real.